



FACULTAD DE INGENIERÍA

Carrera de **INGENIERÍA DE SISTEMAS COMPUTACIONALES**

**“IMPLEMENTACIÓN DE UNA PÁGINA WEB
PARA EL CONTROL DE PRODUCTOS
DISTRIBUIDOS EN EL CHIFA TAT BAO”**

Curso:

**SOLUCIONES WEB Y
APLICACIONES DISTRIBUIDAS**

Autores:

1. Casahuaman Espinoza Javier Oscar
2. Jesús Medina Santiago Alejandro
3. Villegas Huaroc Erick Luis
4. Coveñas Astudillo Jose Andres

Docente:

Lima - Perú

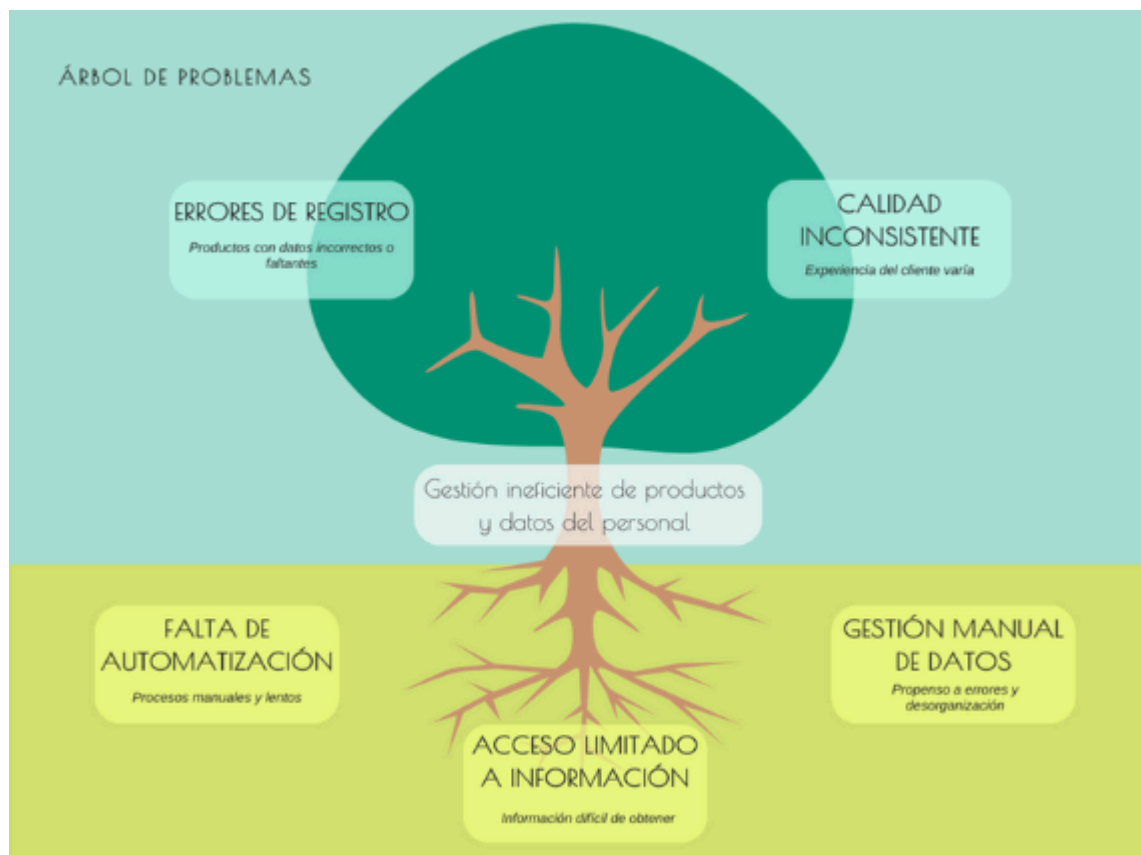
2024

1. Realidad problemática:

Actualmente, el Chifa "Tat Bao", ubicada en el distrito de Miraflores, Lima, enfrenta desafíos operativos significativos derivados de la falta de integración entre sus sistemas y la gestión ineficiente de la administración de productos ofrecidos y datos del personal. Esta situación ha generado una serie de problemas operativos que afectan directamente la experiencia del cliente, como retrasos en la entrega de pedidos, errores en la toma de pedidos y una percepción inconsistente de calidad y servicio en los diferentes puntos de contacto con la empresa, tanto en línea como en sus locales físicos.

2. Árbol de problemas:

Al realizar un análisis más detallado, se identifican diversas causas subyacentes a los problemas operativos mencionados. La falta de especificar datos básicos de los productos ha resultado en una ejecución propensa a errores, mientras que la falta de acceso rápido a la información relevante dificulta la toma de decisiones eficientes. Además, la gestión manual de datos ha demostrado ser insostenible en un entorno operativo dinámico, lo que ha contribuido a la aparición de problemas de calidad y servicio.



3. Descripción de la empresa:

"El Chifa Tat Bao" es una empresa familiar establecida en Lima desde hace más de dos décadas. Su compromiso con la autenticidad de la comida china y la atención al cliente ha ganado la confianza y lealtad de una amplia base de clientes en el distrito de Miraflores y sus alrededores. Con una ubicación estratégica en el corazón de la ciudad, el restaurante se ha convertido en un destino popular tanto para los residentes locales como para los turistas que buscan una experiencia culinaria auténtica.

4. Objetivo General:

El objetivo principal de este proyecto es crear un sistema integral que optimice y registre la variedad de productos y datos del personal, logrando reducir los tiempos de respuesta en un 50%, minimizar los errores en la toma y entrega de pedidos en un 40%, y mejorar la satisfacción del cliente en un 30% en los próximos seis meses. Este sistema permitirá una administración más eficiente y precisa del registro, centralizando y automatizando la información clave, facilitando la toma de decisiones informadas y contribuyendo a la sostenibilidad y crecimiento del negocio.

5. Objetivos específicos:

1. Implementar un sistema automatizado del registro de productos:
Desarrollar un módulo dentro de la página web que permita al personal añadir productos, con una reducción del 50% en errores al registrar datos incorrectamente en los primeros tres meses gracias a la validación.
2. Mejorar el acceso a información relevante para la toma de decisiones:
Crear un dashboard accesible desde la interfaz web que proporcione detalles completos de todos los productos, como precio y tipo, logrando que el 100% del personal pueda acceder y utilizar esta información de manera efectiva dentro del primer mes de implementación.
3. Aumentar la cantidad de personas autorizadas para añadir usuarios y productos:
Al tener una interfaz simple de entender, aseguramos que el 100% del personal esté capacitado para poder registrar nuevos datos sin necesidad de tener conocimientos básicos de Excel.

6. Solución:

Propuesta:

Desarrollar una página web utilizando Angular para la interfaz de administrador, Spring Boot para el backend y una base de datos MySQL. La solución incluirá una API REST para la comunicación entre el frontend y el backend, permitiendo la automatización del registro y visualización de productos.

Sustento:

La integración de Angular y Spring Boot permite una separación clara entre la lógica de presentación y la lógica de negocio, facilitando el mantenimiento y escalabilidad del sistema. MySQL Workbench proporciona una solución robusta para la gestión de datos, asegurando integridad y rapidez en el acceso a la información. La API REST asegurará una comunicación eficiente y segura entre los distintos componentes del sistema.

7. Base de datos:

La base de datos está diseñada para gestionar la operación del Chifa "Tat Bao", estructurada en varias tablas relacionadas entre sí para organizar la información del personal y de los productos.

Usuario:

idusuario: Identificador único del usuario.

nombres: Nombres del personal

apellidos: Apellidos del personal

email: Correo electrónico del usuario.

clave: Contraseña del usuario.

Productos:

idproducto: Identificador único del producto.

nombre: Nombre del producto.

tipo: Especificar si es un plato de entrada, de fondo o bebida.

precio: Precio del producto.

8. Solución API – REST y Spring Boot:

Para el desarrollo del spring decidimos enfocarlo en un patrón MVC. Empezamos con la declaración de nuestros modelos en este caso “Usuarios” haciendo referencia al personal del local, y “Productos” haciendo referencia a todo lo que nos ofrece el local.

```
1 package com.chifa_app.web.models;
2
3 import jakarta.persistence.Entity;
4
5 @Entity
6 @Table(name="usuarios")
7 public class Usuarios {
8     @Id
9     @GeneratedValue(strategy=GenerationType.IDENTITY)
10    private Integer id;
11    private String nombres;
12    private String apellidos;
13    private String correo;
14    private String clave;
15    public Usuarios() {
16    }
17    public Usuarios(String nombres, String apellidos, String correo, String clave) {
18        super();
19        this.nombres = nombres;
20        this.apellidos = apellidos;
21        this.correo = correo;
22        this.clave = clave;
23    }
24    public Integer getId() {
25        return id;
26    }
27    public void setId(Integer id) {
28        this.id = id;
29    }
30    public String getNombres() {
31        return nombres;
32    }
33    public void setNombres(String nombres) {
34        this.nombres = nombres;
35    }
36    public String getApellidos() {
37        return apellidos;
38    }
39    public void setApellidos(String apellidos) {
40        this.apellidos = apellidos;
41    }
42    public String getCorreo() {
43        return correo;
44    }
45    public void setCorreo(String correo) {
46        this.correo = correo;
47    }
48    public String getClave() {
49        return clave;
50    }
51    public void setClave(String clave) {
52        this.clave = clave;
53    }
54 }
55 }
```

Imagen para el modelo de Usuarios

```
1 @Entity
2 @Table(name="productos")
3 public class Productos {
4     @Id
5     @GeneratedValue(strategy=GenerationType.IDENTITY)
6     private Integer id;
7     private String nombre;
8     private String tipo;
9     private Integer precio;
10    public Productos() {
11        super();
12    }
13    public Productos(Integer id, String nombre, String tipo, Integer precio) {
14        super();
15        this.id = id;
16        this.nombre = nombre;
17        this.tipo = tipo;
18        this.precio = precio;
19    }
20    public Integer getId() {
21        return id;
22    }
23    public void setId(Integer id) {
24        this.id = id;
25    }
26    public String getNombre() {
27        return nombre;
28    }
29    public void setNombre(String nombre) {
30        this.nombre = nombre;
31    }
32    public String getTipo() {
33        return tipo;
34    }
35    public void setTipo(String tipo) {
36        this.tipo = tipo;
37    }
38    public Integer getPrecio() {
39        return precio;
40    }
41    public void setPrecio(Integer precio) {
42        this.precio = precio;
43    }
44 }
```

Imagen para el modelo de Productos

Posee también un apartado de DAO (Data Access Object) para la fácil conexión entre nuestro spring y una base de datos. Usando JPARepository para ayudarnos con los queries MySQL y poder hacer múltiples de estos. Optamos por la opción de JPA en lugar del CRUD debido a que suele relacionarse mejor con el spring.

```
1 package com.chifa_app.web.dao;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5
6
7
8
9 public interface IProductosDao extends JpaRepository < Productos, Integer>{
10
11 }
12
```

JPA para Productos

```
1 package com.chifa_app.web.dao;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5
6 public interface IUsuariosDao extends JpaRepository< Usuarios, Integer> {
7
8
9
10 }
11
```

JPA para Usuarios

Un apartado de controladores que incluye nuestros métodos a usar tales como registrar, guardar, eliminar y encontrar por medio del Id.

```
1 @GetMapping("/listarUsuarios")
2 @CrossOrigin
3 public ResponseEntity<List<Usuarios>> obtenerTodosLosUsuarios() {
4     List<Usuarios> usuarios = usuarioService.obtenerTodosLosUsuarios();
5     return new ResponseEntity<>(usuarios, HttpStatus.OK);
6 }
7
8 @DeleteMapping("/eliminarUsuario/{id}")
9 @CrossOrigin
10 public ResponseEntity<String> eliminarUsuarioPorId(@PathVariable("id") Integer id) {
11     Optional<Usuarios> usuarioOptional = usuarioService.obtenerUsuarioPorId(id);
12     if (usuarioOptional.isPresent()) {
13         Usuarios usuario = usuarioOptional.get();
14         usuarioService.eliminarUsuarioPorId(id);
15         String mensaje = "Usuario " + usuario.getNombres() + " " + usuario.getApellidos() + " eliminado correctamente";
16         return new ResponseEntity<>(mensaje, HttpStatus.OK);
17     } else {
18         return new ResponseEntity<>("Usuario no encontrado", HttpStatus.NOT_FOUND);
19     }
20 }
21
22 @CrossOrigin
23 @PostMapping("/registrarUsuario")
24 public ResponseEntity<String> registrarNuevoUsuario(@RequestBody Usuarios usuario) {
25     try {
26         Usuarios usuarioGuardado = usuarioService.registroNuevoUsuarioService(usuario);
27         String mensaje = "Usuario: " + usuarioGuardado.getNombres() + " " + usuarioGuardado.getApellidos() + " registrado exitosamente";
28         return new ResponseEntity<>(mensaje, HttpStatus.OK);
29     } catch (Exception e) {
30         return new ResponseEntity<>("Error al registrar usuario: " + e.getMessage(), HttpStatus.INTERNAL_SERVER_ERROR);
31     }
32 }
33
34 @CrossOrigin
35 @PutMapping("/editarUsuario/{id}")
36 @CrossOrigin
37 public ResponseEntity<String> editarUsuario(@PathVariable("id") Integer id, @RequestBody Usuarios usuario) {
38     Optional<Usuarios> usuarioOptional = usuarioService.obtenerUsuarioPorId(id);
39     if (usuarioOptional.isPresent()) {
40         Usuarios usuarioExistente = usuarioOptional.get();
41         usuarioExistente.setNombres(usuario.getNombres());
42         usuarioExistente.setApellidos(usuario.getApellidos());
43     }
44 }
```

Controlador de Usuarios

```
1 @RestController
2 @RequestMapping("")
3 public class ProductoApiController {
4
5     @Autowired
6     ProductoService productoService;
7
8     @GetMapping("/listarProductos")
9     @CrossOrigin
10     public ResponseEntity<List<Productos>> obtenerTodosLosProductos() {
11         List<Productos> productos = productoService.obtenerTodosLosProductos();
12         return new ResponseEntity<>(productos, HttpStatus.OK);
13     }
14
15     @DeleteMapping("/eliminarProducto/{id}")
16     @CrossOrigin
17     public ResponseEntity<String> eliminarProductoPorId(@PathVariable("id") Integer id) {
18         Optional<Productos> productoOptional = productoService.obtenerProductoPorId(id);
19         if (productoOptional.isPresent()) {
20             Productos producto = productoOptional.get();
21             productoService.eliminarProductoPorId(id);
22             String mensaje = "Producto " + producto.getNombre() + " eliminado correctamente";
23             return new ResponseEntity<>(mensaje, HttpStatus.OK);
24         } else {
25             return new ResponseEntity<>("Producto no encontrado", HttpStatus.NOT_FOUND);
26         }
27     }
28
29     @CrossOrigin
30     @PostMapping("/registrarProducto")
31     public ResponseEntity<String> registrarNuevoProducto(@RequestBody Productos producto) {
32         try {
33             Productos productoGuardado = productoService.registroNuevoProductoService(producto);
34             String mensaje = "Producto: " + productoGuardado.getNombre() + " registrado exitosamente.";
35             return new ResponseEntity<>(mensaje, HttpStatus.OK);
36         } catch (Exception e) {
37             return new ResponseEntity<>("Error al registrar producto: " + e.getMessage(), HttpStatus.INTERNAL_SERVER_ERROR);
38         }
39     }
40 }
```

Controlador de Productos

Y por último un apartado de service donde nos ayudará a llamar a todos nuestros controladores.

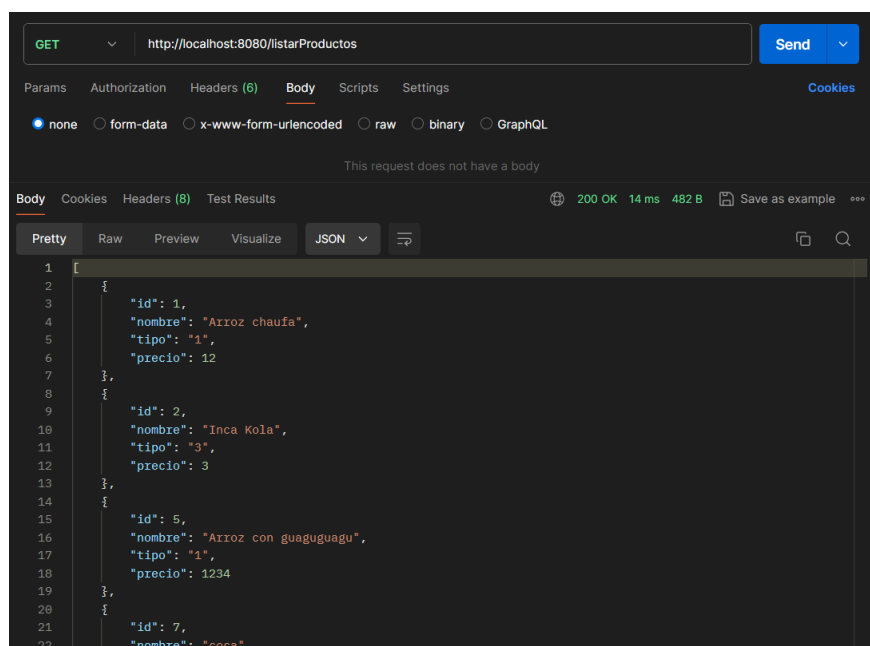
```
4
5 import java.util.List;
6
7
8
9
10
11
12 @Service
13 public class ProductoService {
14     @Autowired
15     IProductosDao productoDao;
16     public Productos registroNuevoProductoService(Productos producto) {
17         return productoDao.save(producto);
18     }
19     public List<Productos> obtenerTodosLosProductos() {
20         return productoDao.findAll();
21     }
22     public Optional<Productos> obtenerProductoPorId(Integer id) {
23         return productoDao.findById(id);
24     }
25
26     public void eliminarProductoPorId(Integer id) {
27         productoDao.deleteById(id);
28     }
29 }
```

Service de Productos

```
1
2
3
4 @Service
5 public class UsuarioService {
6     @Autowired
7     IUsuariosDao usuarioDao;
8
9     public Usuarios registroNuevoUsuarioService(Usuarios usuario) {
10         return usuarioDao.save(usuario);
11     }
12
13     public List<Usuarios> obtenerTodosLosUsuarios() {
14         return usuarioDao.findAll();
15     }
16
17     public Optional<Usuarios> obtenerUsuarioPorId(Integer id) {
18         return usuarioDao.findById(id);
19     }
20
21     public void eliminarUsuarioPorId(Integer id) {
22         usuarioDao.deleteById(id);
23     }
24 }
```

Service de usuarios

Con la ayuda de Postman interactuamos con nuestro spring para poder verificar su correcto funcionamiento.



Postman a lista de productos

9. FrontEnd

I. Componentes

- > editar-producto
- > editar-usuario
- > home
- > layout
- > productos
- > registrar-producto
- > registrar-usuario
- > usuario

a. Layout:

```
layout.component.html X
src > app > layout > layout.component.html > div
Go to component
1 <nav class="navbar navbar-expand-lg bg-body-tertiary">
2   <div class="container-fluid">
3     <a class="navbar-brand" href="/">Chifa Tat Bao</a>
4     <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarSupportedContent"
5       aria-controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle navigation">
6       <span class="navbar-toggler-icon"></span>
7     </button>
8     <div class="collapse navbar-collapse" id="navbarSupportedContent">
9       <ul class="navbar-nav me-auto mb-2 mb-lg-0">
10        <li class="nav-item">
11          <a class="nav-link active" aria-current="page" href="/home">Inicio</a>
12        </li>
13        <li class="nav-item">
14          <a class="nav-link" href="/usuario">Usuarios</a>
15        </li>
16        <li class="nav-item">
17          <a class="nav-link" href="/productos">Productos</a>
18        </li>
19      </ul>
20    </div>
21  </div>
22 </nav>
23 </div></div>
```

b. Home:

```
home.component.html X
src > app > home > home.component.html > div#carouselExampleInterval.carousel.slide
Go to component
1 <app-layout></app-layout>
2
3 <div id="carouselExampleInterval" class="carousel slide" data-bs-ride="carousel">
4   <div class="carousel-inner">
5     <div class="carousel-item active" data-bs-interval="200">
6       
8     </div>
9     <div class="carousel-item" data-bs-interval="200">
10      
12    </div>
13    <div class="carousel-item">
14      
16    </div>
17  </div>
18  <button class="carousel-control-prev" type="button" data-bs-target="#carouselExampleInterval" data-bs-slide="prev">
19    <span class="carousel-control-prev-icon" aria-hidden="true"></span>
20    <span class="visually-hidden">Previous</span>
21  </button>
22  <button class="carousel-control-next" type="button" data-bs-target="#carouselExampleInterval" data-bs-slide="next">
23    <span class="carousel-control-next-icon" aria-hidden="true"></span>
24    <span class="visually-hidden">Next</span>
25  </button>
26 </div>
```


TS home.component.ts X

src > app > home > TS home.component.ts > ...

```
1 import { Component } from '@angular/core';
2 import { LayoutComponent } from '../layout/layout.component'
3
4 @Component({
5   selector: 'app-home',
6   standalone: true,
7   imports: [LayoutComponent],
8   templateUrl: './home.component.html',
9   styleUrls: ['./home.component.css']
10 })
```

c. Usuario:

usuario.component.html X

src > app > usuario > usuario.component.html > div.container.mt-5 > div.row > div.col-lg-8.offset-lg-2 > div.card

Go to component

```
1 <app-layout></app-layout>
2
3 <div class="container mt-5">
4   <div class="row">
5     <div class="col-lg-8 offset-lg-2">
6       <div class="card">
7         <div class="card-body">
8           <span class="titulo">LISTADO DE USUARIOS</span>
9           <button class="btn btn-success float-end" routerLink="/registrar-usuario">CREAR USUARIO</button>
10          <table class="table table-sm table-striped mt-3">
11            <thead>
12              <tr>
13                <th scope="col">Id</th>
14                <th scope="col">Nombres</th>
15                <th scope="col">Apellidos</th>
16                <th scope="col">Correo</th>
17                <th scope="col">Clave</th>
18                <th scope="col"></th>
19              </tr>
20            </thead>
21            <tbody>
22              <tr *ngFor="let usu of datos">
23                <td>{{usu.id}}</td>
24                <td>{{usu.nombres}}</td>
25                <td>{{usu.apellidos}}</td>
26                <td>{{usu.correo}}</td>
27                <td>{{usu.clave}}</td>
28                <td>
29                  <i (click)="editarUsuario(usu.id)" class="fas fa-edit text-primary cursor-pointer"></i>
30                  <i (click)="EliminarUsuario(usu.id)" class="fas fa-trash text-danger cursor-pointer"></i>
31                </td>
32              </tr>
33            </tbody>
34          </table>
35          <button (click)="ListarUsuario()" class="btn btn-primary float-start">Actualizar datos</button>
36        </div>
37      </div>
38    </div>
39  </div>
```

TS usuario.component.ts X

src > app > usuario > TS usuario.component.ts > UsuarioComponent

```
1 import { Component, OnInit } from '@angular/core';
2 import { LayoutComponent } from '../layout/layout.component';
3 import { UsuarioService } from '../usuario-service.service';
4 import { CommonModule } from '@angular/common';
5 import { RouterModule, Router } from '@angular/router';
6 import { ReactiveFormsModule, FormGroup, FormBuilder, Validators, FormsModule } from '@angular/forms';
7
8 @Component({
9   selector: 'app-usuario',
10  standalone: true,
11  imports: [LayoutComponent, CommonModule, RouterModule, ReactiveFormsModule, FormsModule],
12  templateUrl: './usuario.component.html',
13  styleUrls: ['./usuario.component.css']
14 })
15 export class UsuarioComponent implements OnInit {
16
17   datos: any[] = [];
18
19   constructor(private api: UsuarioService, private router: Router) { }
20
21   ngOnInit(): void {
22     this.ListarUsuario();
23   }
24
25   ListarUsuario(): void {
26     this.api.getListarUsuario().subscribe(data => {
27       this.datos = data;
28     });
29   }
30
31   editarUsuario(id: number): void {
32     this.router.navigate(['/editar-usuario', id]);
33   }
34
35   EliminarUsuario(id: number) {
36     if (confirm("¿Está seguro de que desea eliminar este usuario?")) {
37       this.api.DeleteEliminarUsuario(id).subscribe({
38         next: () => {
39           this.ListarUsuario();
40         },
41         error: (error) => {
42           console.error('Error al eliminar usuario:', error);
43         },
44         complete: () => {
45           console.log("Usuario eliminado correctamente");
46         }
47       });
48     }
49   }
50 }
```

d. Registrar-usuario:

```
registrar-usuario.component.html X
src > app > registrar-usuario > registrar-usuario.component.html > div.container.mt-5 > div.row
Go to component
1 <app-layout></app-layout>
2
3 <div class="container mt-5">
4   <div class="row">
5     <div class="col-lg-6 offset-lg-3">
6       <div class="card">
7         <div class="card-body text-center">
8           <span class="titulo">REGISTRAR USUARIO</span>
9           <form class="mt-3 text-start" [formGroup]="usuarioForm" (ngSubmit)="onSubmit()">
10             <div class="mb-3">
11               <input type="text" formControlName="nombres" class="form-control form-control-lg"
12                 placeholder="Nombres">
13               <div *ngIf="usuarioForm.get('nombres')?.invalid && usuarioForm.get('nombres')?.touched"
14                 class="text-danger">
15                 <span>El nombre es<strong> obligatorio</strong></span>
16             </div>
17           </div>
18           <div class="mb-3">
19             <input type="text" formControlName="apellidos" class="form-control form-control-lg"
20               placeholder="Apellidos">
21             <div *ngIf="usuarioForm.get('apellidos')?.invalid && usuarioForm.get('apellidos')?.touched"
22               class="text-danger">
23               <span>El apellido es<strong> obligatorio</strong></span>
24             </div>
25           </div>
26           <div class="mb-3">
27             <input type="email" formControlName="correo" class="form-control form-control-lg"
28               placeholder="Correo">
29             <div *ngIf="usuarioForm.get('correo')?.invalid && usuarioForm.get('correo')?.touched"
30               class="text-danger">
31               <span>El correo es<strong> obligatorio</strong></span>
32             </div>
33           </div>
34           <div class="mb-3">
35             <input type="password" formControlName="clave" class="form-control form-control-lg"
36               placeholder="Contraseña">
37             <div *ngIf="usuarioForm.get('clave')?.invalid && usuarioForm.get('clave')?.touched"
38               class="text-danger">
39               <span>La contraseña es<strong> obligatoria</strong></span>
40             </div>
41           </div>
42           <div class="mb-3">
43             <button routerLink="/usuario" class="btn btn-secondary btn-lg float-start">VOLVER</button>
44             <button type="submit" class="btn btn-success btn-lg float-end">ACEPTAR</button>
45           </div>
46         </form>
47       </div>
48     </div>
49   </div>
50 </div>
```

TS registrar-usuario.component.ts X

src > app > registrar-usuario > TS registrar-usuario.component.ts > ...

```
1 import { Component, OnInit } from '@angular/core';
2 import { LayoutComponent } from '../layout/layout.component';
3 import { UsuarioServiceService } from '../usuario-service.service';
4 import { CommonModule } from '@angular/common';
5 import { RouterModule, Router } from '@angular/router';
6 import { ReactiveFormsModule, FormGroup, FormBuilder, Validators, FormsModule } from '@angular/forms';
7
8 @Component({
9   selector: 'app-registrar-usuario',
10  standalone: true,
11  imports: [LayoutComponent, CommonModule, RouterModule, ReactiveFormsModule, FormsModule],
12  templateUrl: './registrar-usuario.component.html',
13  styleUrls: ['./registrar-usuario.component.css']
14 })
15 export class RegistrarUsuarioComponent implements OnInit {
16
17   usuarioForm!: FormGroup;
18
19   constructor(
20     private fb: FormBuilder,
21     private api: UsuarioServiceService,
22     private router: Router
23   ) {}
24
25   ngOnInit(): void {
26     this.usuarioForm = this.fb.group({
27       nombres: ['', Validators.required],
28       apellidos: ['', Validators.required],
29       correo: ['', [Validators.required, Validators.email]],
30       clave: ['', Validators.required]
31     });
32   }
33
34   onSubmit(): void {
35     if (this.usuarioForm.valid) {
36       this.api.PostRegistrarUsuario(this.usuarioForm.value).subscribe({
37         next: () => {
38           this.router.navigate(['/usuario']);
39         },
40         error: (error) => {
41           console.error('Error al registrar usuario:', error);
42         }
43       });
44     } else {
45       console.log('El formulario no es válido');
46     }
47   }
48 }
```

e. Editar-usuario:

```
editar-usuario.component.html X
src > app > editar-usuario > editar-usuario.component.html > ...
Go to component
1 <app-layout></app-layout>
2
3 <div class="container mt-5">
4   <div class="row">
5     <div class="col-lg-6 offset-lg-3">
6       <div class="card">
7         <div class="card-body text-center">
8           <span class="titulo">EDITAR USUARIO</span>
9           <form class="mt-3 text-start" [formGroup]="usuarioForm" (ngSubmit)="onSubmit()">
10             <div class="mb-3">
11               <input type="text" formControlName="nombres" class="form-control form-control-lg" placeholder="Nombres">
12               <div *ngIf="usuarioForm.get('nombres')?.invalid && usuarioForm.get('nombres')?.touched" class="text-danger">
13                 <span>El nombre es obligatorio</span>
14               </div>
15             </div>
16             <div class="mb-3">
17               <input type="text" formControlName="apellidos" class="form-control form-control-lg" placeholder="Apellidos">
18               <div *ngIf="usuarioForm.get('apellidos')?.invalid && usuarioForm.get('apellidos')?.touched" class="text-danger">
19                 <span>El apellido es obligatorio</span>
20               </div>
21             </div>
22             <div class="mb-3">
23               <input type="email" formControlName="correo" class="form-control form-control-lg" placeholder="Correo">
24               <div *ngIf="usuarioForm.get('correo')?.invalid && usuarioForm.get('correo')?.touched" class="text-danger">
25                 <span>El correo es obligatorio</span>
26               </div>
27             </div>
28             <div class="mb-3">
29               <input type="password" formControlName="clave" class="form-control form-control-lg" placeholder="Contraseña">
30               <div *ngIf="usuarioForm.get('clave')?.invalid && usuarioForm.get('clave')?.touched" class="text-danger">
31                 <span>La contraseña es obligatoria</span>
32               </div>
33             </div>
34             <div class="mb-3">
35               <button routerLink="/usuario" class="btn btn-secondary btn-lg float-start">VOLVER</button>
36               <button type="submit" class="btn btn-success btn-lg float-end">ACEPTAR</button>
37             </div>
38           </form>
39         </div>
40       </div>
41     </div>
42   </div>
43 </div>
```

TS editor-usuario.component.ts X

src > app > editor-usuario > TS editor-usuario.component.ts > EditorUsuarioComponent

```
1  import { Component, OnInit } from '@angular/core';
2  import { LayoutComponent } from '../layout/layout.component';
3  import { UsuarioServiceService } from '../usuario-service.service';
4  import { CommonModule } from '@angular/common';
5  import { RouterModule, Router, ActivatedRoute } from '@angular/router';
6  import { ReactiveFormsModule, FormGroup, FormBuilder, Validators, FormsModule } from '@angular/forms';
7
8  @Component({
9    selector: 'app-editor-usuario',
10   standalone: true,
11   imports: [LayoutComponent, CommonModule, RouterModule, ReactiveFormsModule, FormsModule],
12   templateUrl: './editor-usuario.component.html',
13   styleUrls: ['./editor-usuario.component.css']
14 })
15 export class EditorUsuarioComponent implements OnInit {
16
17   usuarioForm!: FormGroup;
18   usuarioId!: number;
19
20   constructor(
21     private fb: FormBuilder,
22     private api: UsuarioServiceService,
23     private router: Router,
24     private route: ActivatedRoute
25   ) { }
26
27   ngOnInit(): void {
28     this.usuarioForm = this.fb.group({
29       nombres: ['', Validators.required],
30       apellidos: ['', Validators.required],
31       correo: ['', [Validators.required, Validators.email]],
32       clave: ['', Validators.required]
33     });
34
35     this.route.params.subscribe(params => {
36       this.usuarioId = +params['id'];
37       this.cargarUsuario(this.usuarioId);
38     });
39   }
40
41   cargarUsuario(id: number): void {
42     this.api.getUsuarioPorId(id).subscribe({
43       next: (usuario) => {
44         this.usuarioForm.patchValue(usuario);
45       },
46       error: (error) => {
47         console.error('Error al cargar usuario:', error);
48       }
49     });
50   }
51
52   onSubmit(): void {
53     if (this.usuarioForm.valid) {
54       this.api.putEditorUsuario(this.usuarioId, this.usuarioForm.value).subscribe({
55         next: () => {
56           this.router.navigate(['/usuario']);
57         },
58         error: (error) => {
59           console.error('Error al actualizar usuario:', error);
60         }
61       });
62     } else {
63       console.log('El formulario no es válido');
64     }
65   }
66 }
```

f. Productos:

```
products.component.html X
src > app > productos > products.component.html > ...
Go to component
1 <app-layout></app-layout>
2
3 <div class="container mt-5">
4   <div class="row">
5     <div class="col-lg-8 offset-lg-2">
6       <div class="card">
7         <div class="card-body">
8           <span class="titulo">LISTADO DE PRODUCTOS</span>
9           <button class="btn btn-success float-end" routerLink="/registrar-producto">AÑADIR PRODUCTO</button>
10          <table class="table table-sm table-striped mt-3">
11            <thead>
12              <tr>
13                <th scope="col">Id</th>
14                <th scope="col">Nombre</th>
15                <th scope="col">Tipo</th>
16                <th scope="col">Precio</th>
17                <th scope="col"></th>
18              </tr>
19            </thead>
20            <tbody>
21              <tr *ngFor="let prod of datos">
22                <td>{{prod.id}}</td>
23                <td>{{prod.nombre}}</td>
24                <td>{{prod.tipo}}</td>
25                <td>{{prod.precio}}</td>
26                <td>
27                  <i (click)="editarProducto(prod.id)" class="fas fa-edit text-primary cursor-pointer"></i>
28                  <i (click)="EliminarProducto(prod.id)" class="fas fa-trash text-danger cursor-pointer"></i>
29                </td>
30              </tr>
31            </tbody>
32          </table>
33          <button (click)="ListarProducto()" class="btn btn-primary float-start">Actualizar datos</button>
34        </div>
35      </div>
36    </div>
37  </div>
38 </div>
```

TS productos.component.ts X

src > app > productos > TS productos.component.ts > ...

```
1 import { Component, OnInit } from '@angular/core';
2 import { LayoutComponent } from '../layout/layout.component';
3 import { ProductoServiceService } from '../producto-service.service';
4 import { CommonModule } from '@angular/common';
5 import { RouterModule, Router } from '@angular/router';
6 import { ReactiveFormsModule, FormGroup, FormBuilder, Validators, FormsModule } from '@angular/forms';
7 |
8 @Component({
9   selector: 'app-productos',
10  standalone: true,
11  imports: [LayoutComponent, CommonModule, RouterModule, ReactiveFormsModule, FormsModule],
12  templateUrl: './productos.component.html',
13  styleUrls: ['./productos.component.css']
14 })
15
16 export class ProductosComponent implements OnInit {
17
18   datos: any[] = [];
19
20   constructor(private api: ProductoServiceService, private router: Router) { }
21
22   ngOnInit(): void {
23     this.ListarProducto();
24   }
25
26   ListarProducto(): void {
27     this.api.getListarProducto().subscribe(data => {
28       this.datos = data;
29     });
30   }
31
32   editarProducto(id: number): void {
33     this.router.navigate(['/editar-producto', id]);
34   }
35
36   EliminarProducto(id: number) {
37     if (confirm("¿Está seguro de que desea eliminar este producto?")) {
38       this.api.DeleteEliminarProducto(id).subscribe({
39         next: () => {
40           this.ListarProducto();
41         },
42         error: (error) => {
43           console.error('Error al eliminar producto:', error);
44         },
45         complete: () => {
46           console.log("Producto eliminado correctamente");
47         }
48       });
49     }
50   }
51 }
```


g. Registrar-producto:

```
registrar-producto.component.html X
src > app > registrar-producto > registrar-producto.component.html > ...
Go to component
1 <app-layout></app-layout>
2
3 <div class="container mt-5">
4   <div class="row">
5     <div class="col-lg-6 offset-lg-3">
6       <div class="card">
7         <div class="card-body text-center">
8           <span class="titulo">REGISTRAR PRODUCTO</span>
9           <form class="mt-3 text-start" [formGroup]="productoForm" (ngSubmit)="onSubmit()">
10             <div class="mb-3">
11               <input type="text" formControlName="nombre" class="form-control form-control-lg"
12                 placeholder="Nombre">
13               <div *ngIf="productoForm.get('nombre')?.invalid && productoForm.get('nombre')?.touched"
14                 class="text-danger">
15                 <span>El nombre es<strong> obligatorio</strong></span>
16               </div>
17             </div>
18             <div class="mb-3">
19               <input type="text" formControlName="tipo" class="form-control form-control-lg"
20                 placeholder="Tipo">
21               <div *ngIf="productoForm.get('tipo')?.invalid && productoForm.get('tipo')?.touched"
22                 class="text-danger">
23                 <span>El tipo es<strong> obligatorio</strong></span>
24               </div>
25             </div>
26             <div class="mb-3">
27               <input type="text" formControlName="precio" class="form-control form-control-lg"
28                 placeholder="Precio">
29               <div *ngIf="productoForm.get('precio')?.invalid && productoForm.get('precio')?.touched"
30                 class="text-danger">
31                 <span>El precio es<strong> obligatorio</strong></span>
32               </div>
33             </div>
34             <div class="mb-3">
35               <button routerLink="/productos" class="btn btn-secondary btn-lg float-start">VOLVER</button>
36               <button type="submit" class="btn btn-success btn-lg float-end">ACEPTAR</button>
37             </div>
38           </form>
39         </div>
40       </div>
41     </div>
42   </div>
43 </div>
```

registrar-producto.component.html

TS registrar-producto.component.ts X

src > app > registrar-producto > TS registrar-producto.component.ts > ...

```
1  import { Component, OnInit } from '@angular/core';
2  import { LayoutComponent } from '../layout/layout.component';
3  import { ProductoServiceService } from '../producto-service.service';
4  import { CommonModule } from '@angular/common';
5  import { RouterModule, Router } from '@angular/router';
6  import { ReactiveFormsModule, FormGroup, FormBuilder, Validators, FormsModule } from '@angular/forms';
7
8  @Component({
9    selector: 'app-registrar-producto',
10    standalone: true,
11    imports: [LayoutComponent, CommonModule, RouterModule, ReactiveFormsModule, FormsModule],
12    templateUrl: './registrar-producto.component.html',
13    styleUrls: ['./registrar-producto.component.css']
14  })
15
16  export class RegistrarProductoComponent implements OnInit {
17
18    productoForm!: FormGroup;
19
20    constructor(
21      private fb: FormBuilder,
22      private api: ProductoServiceService,
23      private router: Router
24    ) {}
25
26    ngOnInit(): void {
27      this.productoForm = this.fb.group({
28        nombre: ['', Validators.required],
29        tipo: ['', Validators.required],
30        precio: ['', [Validators.required]]
31      });
32    }
33
34    onSubmit(): void {
35      if (this.productoForm.valid) {
36        this.api.PostRegistrarProducto(this.productoForm.value).subscribe({
37          next: () => {
38            this.router.navigate(['/productos']);
39          },
40          error: (error) => {
41            console.error('Error al registrar producto:', error);
42          }
43        });
44      } else {
45        console.log('El formulario no es válido');
46      }
47    }
48  }
```


TS editar-producto.component.ts X

src > app > editar-producto > TS editar-producto.component.ts > ...

```
1 import { Component, OnInit } from '@angular/core';
2 import { LayoutComponent } from '../layout/layout.component';
3 import { ProductoServiceService } from '../producto-service.service';
4 import { CommonModule } from '@angular/common';
5 import { RouterModule, Router, ActivatedRoute } from '@angular/router';
6 import { ReactiveFormsModule, FormGroup, FormBuilder, Validators, FormsModule } from '@angular/forms';
7
8 @Component({
9   selector: 'app-editar-producto',
10  standalone: true,
11  imports: [LayoutComponent, CommonModule, RouterModule, ReactiveFormsModule, FormsModule],
12  templateUrl: './editar-producto.component.html',
13  styleUrls: ['./editar-producto.component.css']
14 })
15
16 export class EditarProductoComponent implements OnInit {
17
18   productoForm!: FormGroup;
19   productoId!: number;
20
21   constructor(
22     private fb: FormBuilder,
23     private api: ProductoServiceService,
24     private router: Router,
25     private route: ActivatedRoute
26   ) { }
27
28   ngOnInit(): void {
29     this.productoForm = this.fb.group({
30       nombre: ['', Validators.required],
31       tipo: ['', Validators.required],
32       precio: ['', Validators.required]
33     });
34
35     this.route.params.subscribe(params => {
36       this.productoId = +params['id'];
37       this.cargarProducto(this.productoId);
38     });
39   }
40
41   cargarProducto(id: number): void {
42     this.api.getProductosPorId(id).subscribe({
43       next: (producto) => {
44         this.productoForm.patchValue(producto);
45       },
46       error: (error) => {
47         console.error('Error al cargar producto:', error);
48       }
49     });
50   }
51
52   onSubmit(): void {
53     if (this.productoForm.valid) {
54       this.api.putEditarProducto(this.productoId, this.productoForm.value).subscribe({
55         next: () => {
56           this.router.navigate(['/productos']);
57         },
58         error: (error) => {
59           console.error('Error al actualizar producto:', error);
60         }
61       });
62     } else {
63       console.log('El formulario no es válido');
64     }
65   }
66 }
```

II. Servicios

```
TS producto-service.service.spec.ts
TS producto-service.service.ts
TS usuario-service.service.spec.ts
TS usuario-service.service.ts
```

TS usuario-service.service.ts X

src > app > TS usuario-service.service.ts > ...

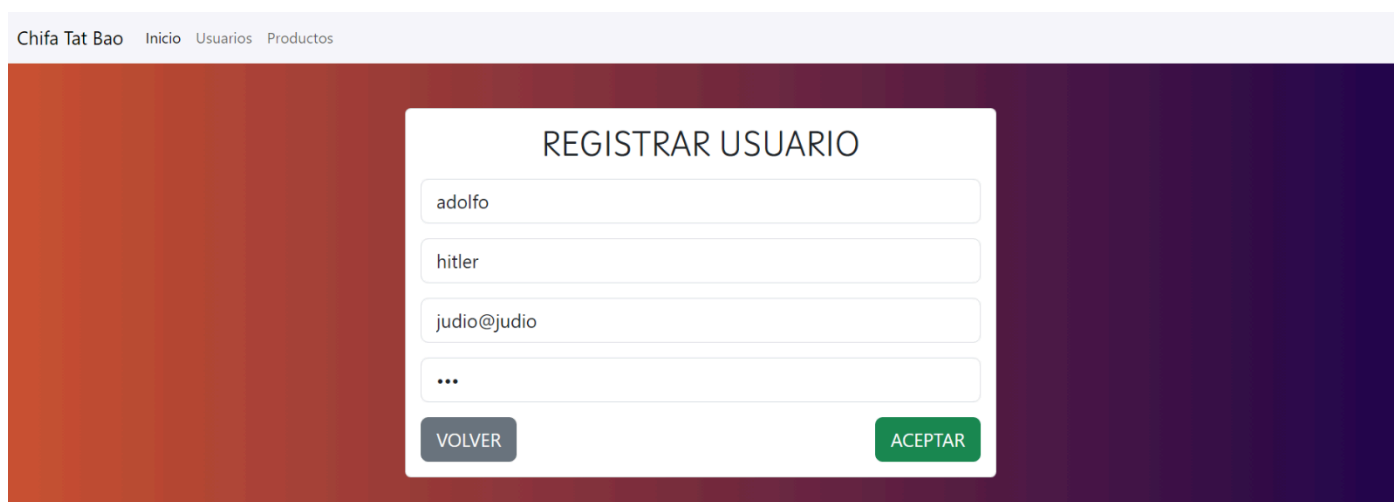
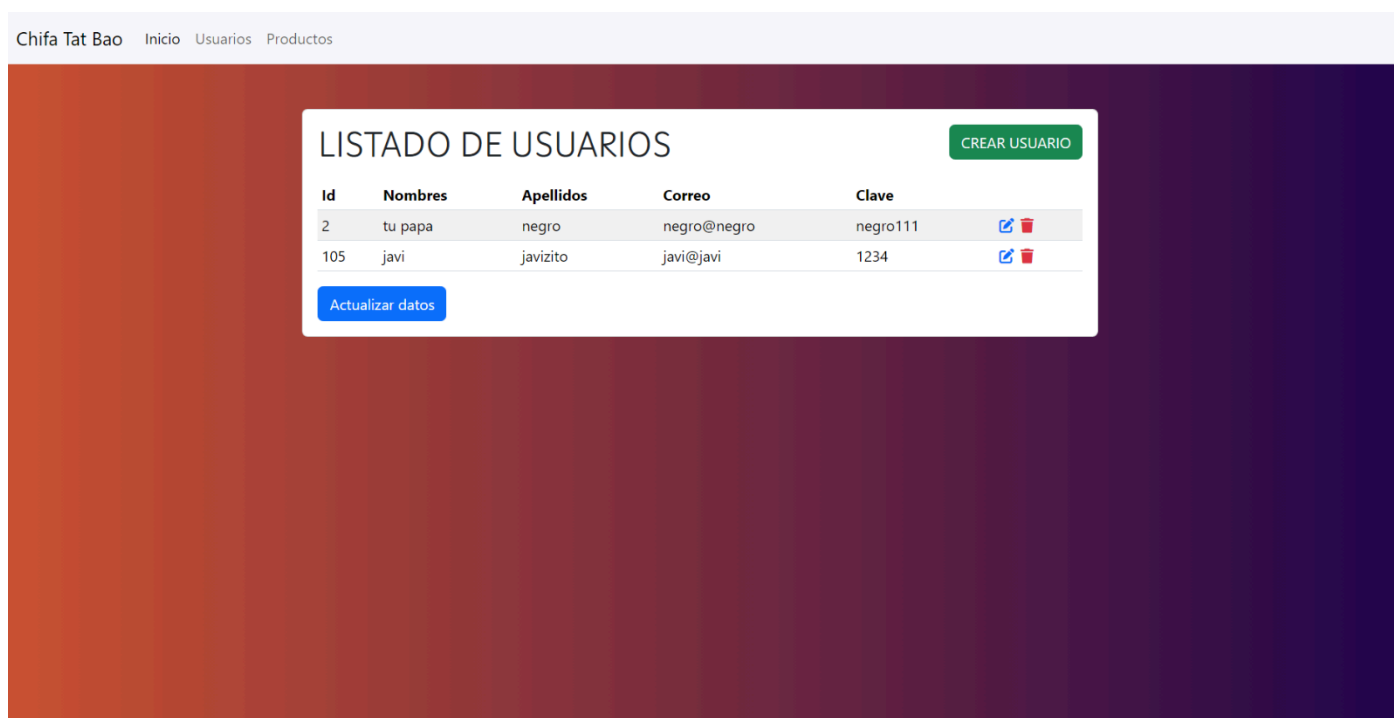
```
1  import { HttpClient } from '@angular/common/http';
2  import { Injectable } from '@angular/core';
3  import { Observable } from 'rxjs';
4
5  @Injectable({
6    providedIn: 'root'
7  })
8  export class UsuarioServiceService {
9
10     private base_url = "http://localhost:8080/";
11
12     constructor(private http: HttpClient) { }
13
14     getListarUsuario(): Observable<any> {
15         return this.http.get(`${this.base_url}listarUsuarios`);
16     }
17
18     PostRegistrarUsuario(datos: any): Observable<any> {
19         return this.http.post(this.base_url + "registrarUsuario", datos);
20     }
21
22     DeleteEliminarUsuario(id: number): Observable<any> {
23         return this.http.delete(`${this.base_url}eliminarUsuario/${id}`);
24     }
25
26     PutEditarUsuario(id: number, datos: any): Observable<any> {
27         return this.http.put(`${this.base_url}editarUsuario/${id}`, datos);
28     }
29
30     getUsuarioPorId(id: number): Observable<any> {
31         return this.http.get(`${this.base_url}usuario/${id}`);
32     }
33 }
```

```
TS usuario-service.service.ts TS producto-service.service.ts X
src > app > TS producto-service.service.ts > ...
1  import { HttpClient } from '@angular/common/http';
2  import { Injectable } from '@angular/core';
3  import { Observable } from 'rxjs';
4
5  @Injectable({
6    providedIn: 'root'
7  })
8  export class ProductoServiceService {
9
10     private base_url = "http://localhost:8080/";
11
12     constructor(private http: HttpClient) { }
13
14     getListaProducto(): Observable<any> {
15         return this.http.get(`${this.base_url}listarProductos`);
16     }
17
18     PostRegistrarProducto(datos: any): Observable<any> {
19         return this.http.post(this.base_url + "registrarProducto", datos);
20     }
21
22     DeleteEliminarProducto(id: number): Observable<any> {
23         return this.http.delete(`${this.base_url}eliminarProducto/${id}`);
24     }
25
26     PutEditarProducto(id: number, datos: any): Observable<any> {
27         return this.http.put(`${this.base_url}editarProducto/${id}`, datos);
28     }
29
30     getProductoPorId(id: number): Observable<any> {
31         return this.http.get(`${this.base_url}producto/${id}`);
32     }
33 }
```

III. Configuración y Rutas

```
TS app.config.ts X TS app.routes.ts
src > app > TS app.config.ts > ...
1  import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
2  import { provideRouter } from '@angular/router';
3
4  import { routes } from './app.routes';
5  import { provideClientHydration } from '@angular/platform-browser';
6  import { provideHttpClient, withFetch } from '@angular/common/http';
7
8  export const appConfig: ApplicationConfig = {
9      providers: [provideHttpClient(withFetch()), provideZoneChangeDetection({ eventCoalescing: true }),
10                 provideRouter(routes), provideClientHydration()]
11  };
12
```

```
TS app.config.ts TS app.routes.ts X
src > app > TS app.routes.ts > ...
1  import { Routes } from '@angular/router';
2  import { HomeComponent } from './home/home.component';
3
4  import { UsuarioComponent } from './usuario/usuario.component';
5  import { RegistrarUsuarioComponent } from './registrar-usuario/registrar-usuario.component';
6  import { EditarUsuarioComponent } from './editar-usuario/editar-usuario.component';
7
8  import { ProductosComponent } from './productos/productos.component';
9  import { RegistrarProductoComponent } from './registrar-producto/registrar-producto.component';
10 import { EditarProductoComponent } from './editar-producto/editar-producto.component';
11
12 export const routes: Routes = [
13   { path: '', component: HomeComponent },
14   { path: 'home', component: HomeComponent },
15   { path: 'usuario', component: UsuarioComponent },
16   { path: 'registrar-usuario', component: RegistrarUsuarioComponent },
17   { path: 'editar-usuario/:id', component: EditarUsuarioComponent },
18   { path: 'productos', component: ProductosComponent },
19   { path: 'registrar-producto', component: RegistrarProductoComponent },
20   { path: 'editar-producto/:id', component: EditarProductoComponent },
21   { path: '**', redirectTo: '', pathMatch: 'full' }
22 ];
```



EDITAR USUARIO

localhost:4200 dice

¿Está seguro de que desea eliminar este usuario?

LISTADO DE USUARIOS

Id	Nombres	Apellidos	Correo	Clave	
2	tu papa	negro	negro@negro	negro111	 
105	javi	javizito	javi@javi	1234	 
106	adolfo	hitler	judio@judio	666	 

LISTADO DE PRODUCTOS

AÑADIR PRODUCTO

Id	Nombre	Tipo	Precio	
1	Arroz chaufa	1	12	 
2	Inca Kola	3	3	 
5	Arroz con guaguguagu	comida	1234	 
7	coca	bebida	100	 

Actualizar datos

REGISTRAR PRODUCTO

coca

bebida

3

VOLVER

ACEPTAR

10. Referencias

- Pérez, J. M., & García, A. (2023). Arquitectura de una aplicación web moderna con Angular, Spring Boot y MySQL. *ReVisión: Revista de Investigaciones Científicas*, 35(2), 235-250. <https://www.flightradar24.com/data/flights/fr4075>
- Toro, I. (2022). Desarrollo de una aplicación web de gestión de proyectos con Angular, Spring Boot y MySQL. Trabajo de Fin de Grado, Universidad Politécnica de Madrid. <https://www.flightradar24.com/data/flights/fr1305>
- López, M., & Fernández, J. (2022). Análisis del rendimiento de una aplicación web con Angular, Spring Boot y MySQL. Memoria, Universitat Politècnica de Catalunya. <https://upcommons.upc.edu/bitstream/handle/2117/179473/memoria.pdf?sequence=1&isAllowed=y>
- Diseño e implementación de una plataforma web para la gestión de cursos online con Angular, Spring Boot y MySQL. Tesis Doctoral, Universidad Nacional de Quilmes. <https://www.unq.edu.ar/>
- Aplicación web para la gestión de registros médicos con Angular, Spring Boot y MySQL. Tesis Doctoral, Universitat Politècnica de Catalunya. <https://serveistic.upc.edu/ca/eres>
- Pérez, J. M., & García, A. (2023). Implementación de un sistema de recomendación de productos con Angular, Spring Boot y MySQL. *ReVisión: Revista de Investigaciones Científicas*, 35(2), 235-250. <https://www.flightradar24.com/data/flights/fr4075>